Consistent Hashing

為什麼需要一致性雜湊 (Consistent Hashing)？

在分散式系統中，資料通常需要分散到多台伺服器上。

最直觀的方法是用 **簡單 hash**：

```
server = hash(key) % N    # N = 節點數
```

問題是：

- 如果 N 改變（新增或移除伺服器），幾乎所有資料都需要重新分配。
- 這會造成大量搬遷，導致 **效能問題** 與 **系統不穩定**。

```
運算規則：value % N = Node Number
例如：3 % 3 = 0 -> Node 0

N = 3, 假設我們有 1 到 9 的值

3, 6, 9 -> Node 0
1, 4, 7 -> Node 1
2, 5, 8 -> Node 2

如果這邊多加一個 node -> N = 4

4, 8 -> Node 0
1, 5, 9 -> Node 1
2, 6 -> Node 2
3, 7 -> Node 3

→ 9 個值裡有 7 個值被移動（只有 1 和 2 留在原位）
```

Consistent Hashing 的核心價值：

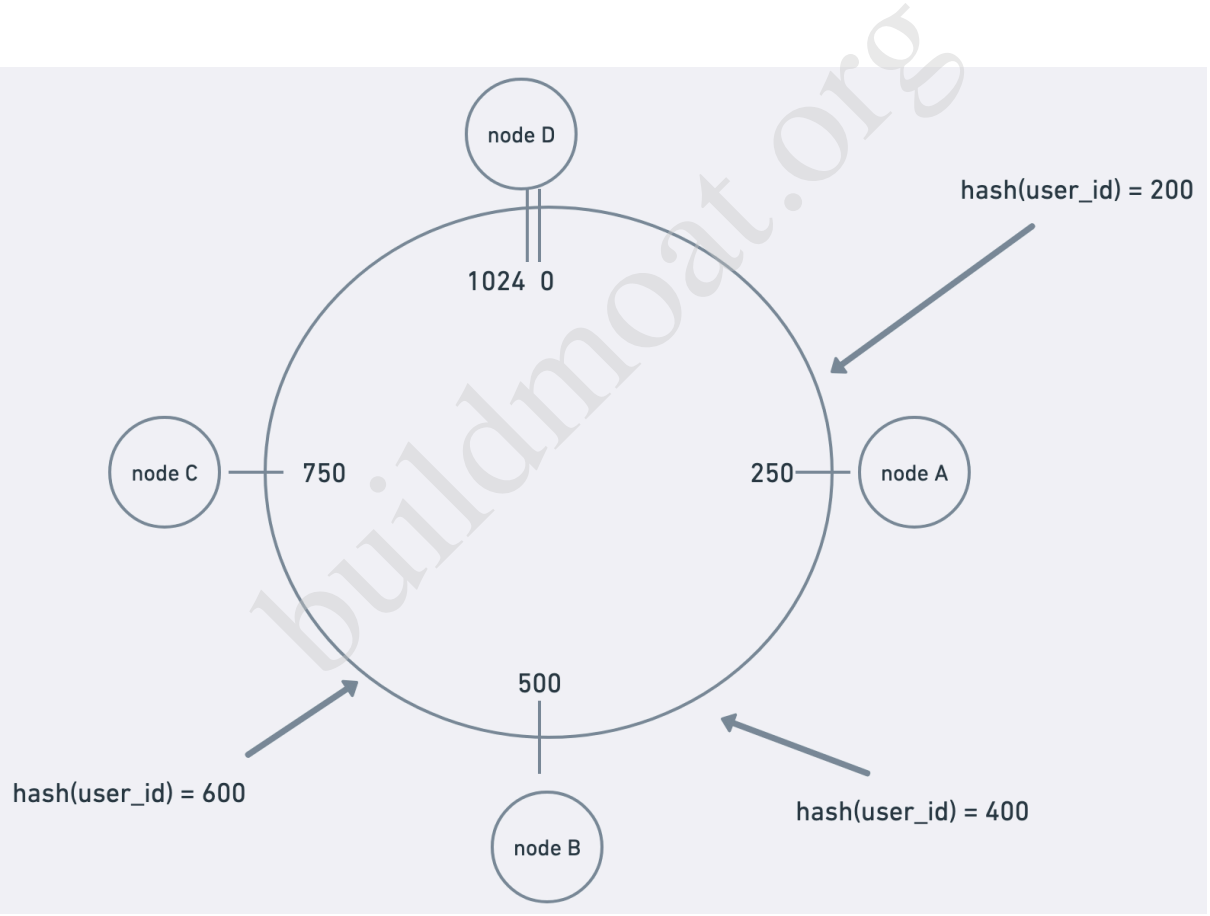
buildmoat.org

當伺服器數量改變時，只需要搬動一小部分資料，大部分資料仍然留在原來的節點。

Consistent Hashing 怎麼運作？

(a) Hash 環 (Hash Ring)

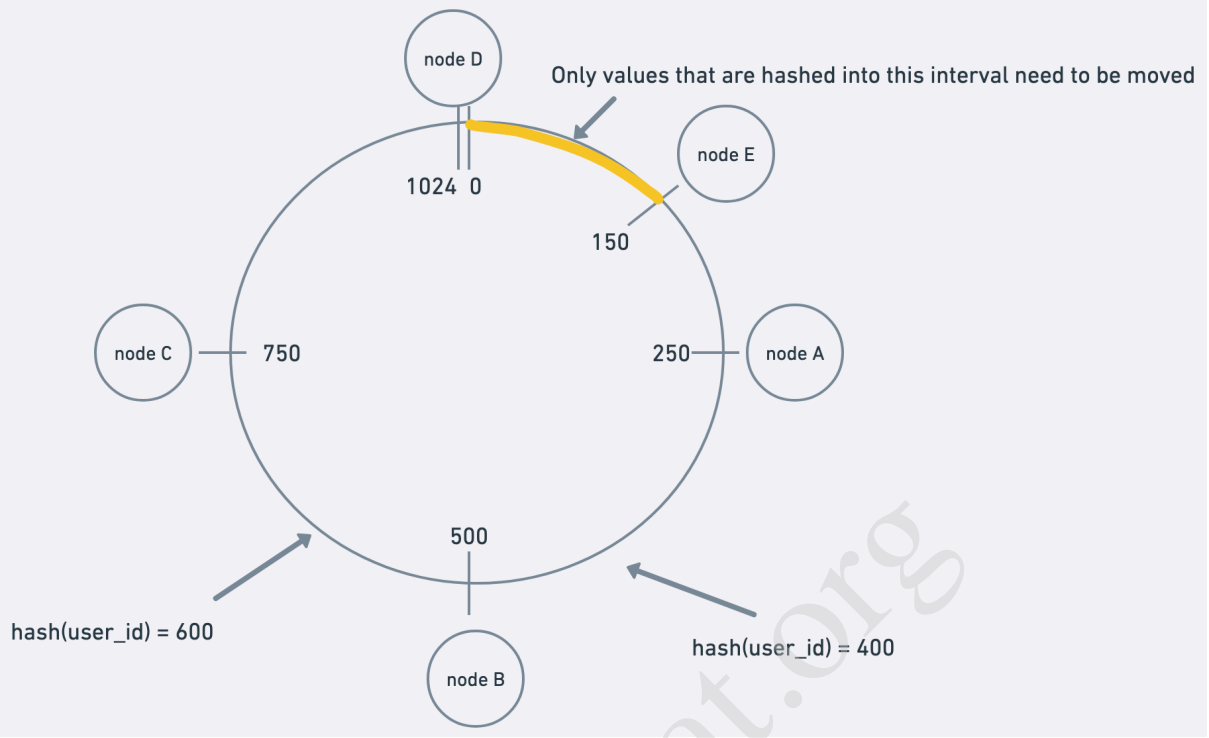
- 把所有可能的 hash 值映射到一個圓環上（0 → MAX_HASH）。
- 每個節點 (server) 用 hash 函數映射到圓環上的一個位置。
- 每筆資料也用 hash(key) 計算位置。
- **規則**：資料順時針找到第一個節點，交給它存。



buildmoat.org

- hashed value = 200 → 落在 A (0 ~ 250)
- hashed value = 400 → 落在 B (250 ~ 500)
- hashed value = 600 → 落在 C (500 ~ 750)

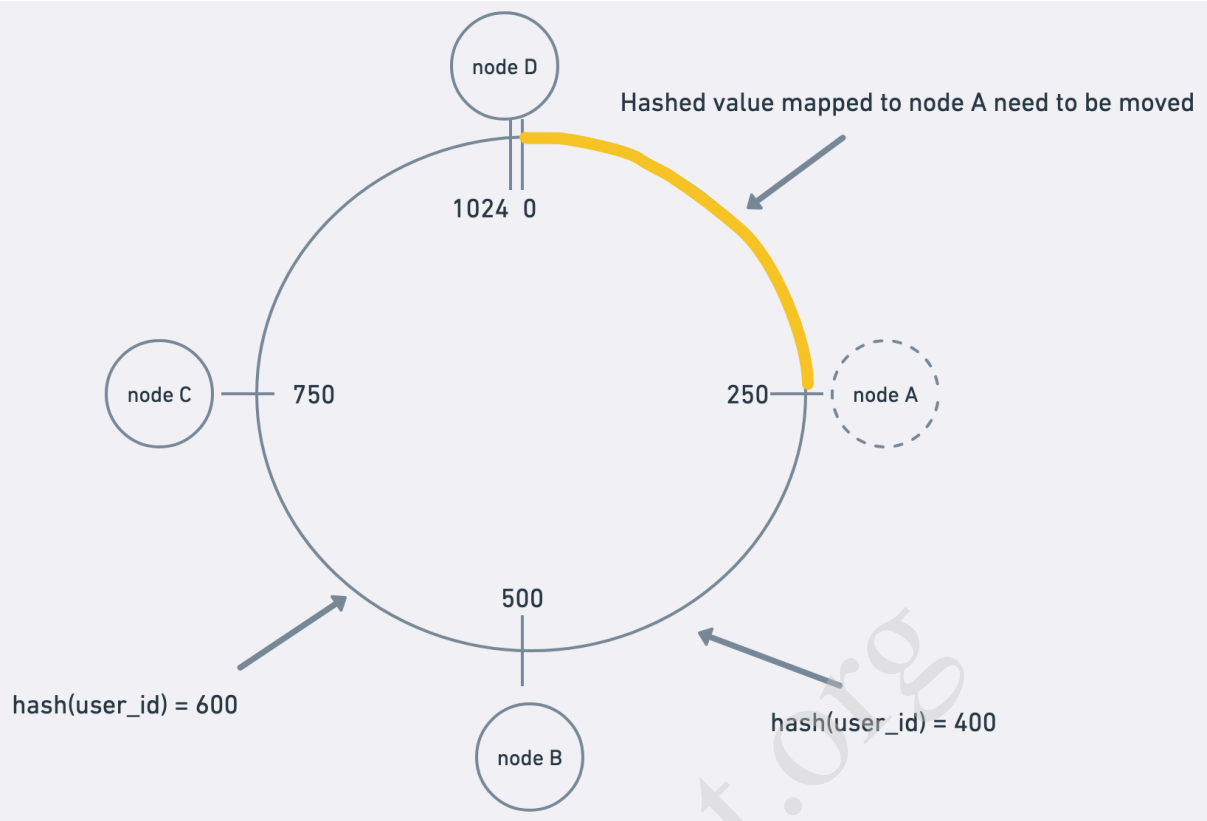
(b) 新節點加入



- 假設新節點 E 落在 150。
- 只有「D 與 E 之間」（0～150）的資料，從 A 搬到 E。
- 其他資料不變。

(c) 節點移除

buildmoat.org



- 假設節點 A 下線。
- 原本屬於 A 的資料會移到下一個節點 B。
- 其他資料不受影響。

虛擬節點 (Virtual Nodes)

1. 為什麼需要虛擬節點？

在 Consistent Hashing 的基本版本裡：

- 每個實體節點 (server) 在 hash 環上只有一個位置。
- 問題：
  1. **資料分佈不均**：hash 雖然隨機，但節點少時，分配可能很不平均。
     - 在上面的例子裡，移除節點 A 以後，節點 B 需要負擔全部一半的 hashed values。
  2. **節點性能不同**：有的伺服器性能強、有的弱，但一樣只拿到一個區間 → 沒有照硬體能力分配。

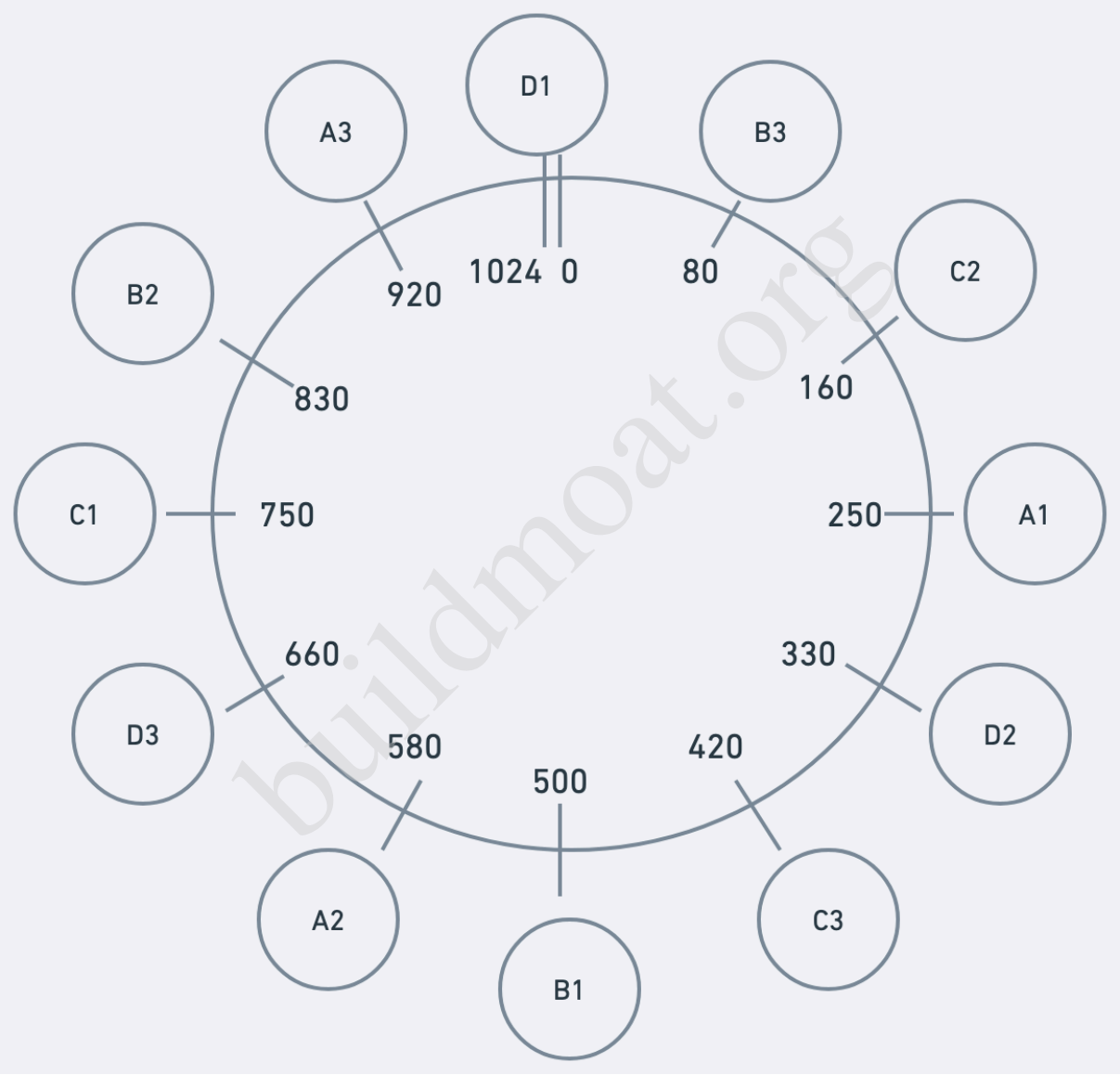
解法：引入虛擬節點。

buildmoat.org

2. 虛擬節點的做法

- 每個實體節點對應到多個「虛擬節點 (vNodes)」。
- vNodes 也會映射到 hash 環上，就像多個小節點。
- 資料依舊依照 **順時針規則** 找到最近的 vNode 存放，然後交由對應的實體節點。

例子：




```
原本：
A(250)   B(500)   C(750)  D(1024)

使用 vNodes：
A1(250), A2(580), A3(920)
```

```
B1(500), B2(830), B3(80)
C1(750), C2(160), C3(420)
D1(1024), D2(330), D3(660)
```

這樣一來：

- A、B、C 各自擁有多個位置。
- key 的分布更隨機、更均勻。

3. 虛擬節點的好處

1. **平均分佈**：即使節點數少，也能把資料更均勻地切開。
2. **依硬體能力分配**：可以讓效能較強的機器放更多 vNodes，負責更多資料；效能較弱的機器放少一點。
3. **彈性擴展**：當節點加入/移除時，因為每個節點有多個 vNodes，搬遷的資料更細顆粒化，負載更平滑。

在 System Design Interview 的應用

什麼時候要想到 Consistent Hashing？

在以下情境，只要「**節點數可能變動**、**要做** sharding/routing、**希望最少搬遷**」，就應優先考慮：

- Sharding for Distributed Cache ：Memcached/Redis Sharding（避免 `key % N` 帶來的大規模重分配）
- Storage sharding/routing：Cassandra/Dynamo 風格的 partition routing
- **Sticky connections for users**：聊天室/即時服務將同一使用者或同一聊天室路由到固定節點
- **API Gateway / Sticky Sessions**：減少跨節點 session 同步
- **Rate Limiting**：把 key（user_id, api_key）穩定地分到對應節點做限流統計
- Metrics / Aggregation：將相同維度（e.g., (user_id, feature)）聚到固定節點做累加，降低跨節點匯總成本
- **CDN/Edge Routing**：把 URL 或內容 ID 穩定映射到 edge 節點做快取

記憶口訣：「變動的節點 + 需要穩定歸屬 + 想少搬家」→ 用 consistent hashing。

buildmoat.org

自我測驗

Q1：簡單 hash（hash(key) % N）在什麼情況下會出問題？用一個例子說明。

> 當節點數 N 改變時（新增或移除伺服器），幾乎所有 key 的 `hash(key) % N` 結果都會改變，導致大量資料需要重新分配。
>
> 例如：N=3 時有 9 個 key 分佈在 3 個節點上。N 變成 4 時，其中 7 個 key 的位置都改變了，需要大量搬遷。

Q2：是非題 — 在 consistent hashing 中新增一個節點時，所有 key 都需要重新分配。

> **錯。** 只有落在新節點與其前一個節點之間的 key 需要搬移到新節點，其他 key 不受影響。這正是 consistent hashing 的核心價值。

Q3：Virtual Node（虛擬節點）解決了哪兩個問題？

> **資料分佈不均**：節點數少時，hash 環上的位置可能集中，導致某些節點負擔過重。虛擬節點讓分佈更均勻。**硬體差異**：效能較強的機器可以分配更多虛擬節點來承擔更多負載，實現按硬體能力分配。

Q4：列舉三個在系統設計面試中適合使用 consistent hashing 的場景。

> 任意三個即可：
>
> - 分散式快取 sharding（Memcached / Redis）
> - 儲存路由（Cassandra / Dynamo 風格）
> - 聊天室 / 即時服務的 sticky connections
> - API Gateway / Sticky Sessions
> - Rate Limiting
> - Metrics / Aggregation
> - CDN / Edge Routing

buildnotes.org